Géométrie des centroïdes et formules exactes dans c2blue55 (v8.2)

Centrage a priori 512D, rotation orthonormale Rademacher-Hadamard RaBitQ 1-bit, rayon conforme, têtes INT8 et garde CUSUM

Principe d'ingénierie (vue d'ensemble)

Sur un socket DNS à haut débit ou un anneau de télémétrie système, exécuter un réseau de neurones profond sur chaque paquet dépasse les budgets de latence à la microseconde et expose le moteur à des allocations imprévisibles sur le tas. `c2blue55` remplace cette inférence lourde par une **cascade déterministe sans allocation** (`0 B/op`, `6,68–11,94 µs/op` avec `GOEXPERIMENT=simd` / plafond scalaire `8,63–16,32 µs/op`, binaire autonome `c2agent` de `7,58 Mio`) adossée à des disquettes vectorielles projetées en lecture seule par `mmap` (`.c2book`, format `C2FLOP1`).

Chaque observation (jusqu'à `4 096` octets) est convertie en un vecteur de `512` flottants, centrée autour d'un point neutre analytique, tournée isométriquement par transformée de Walsh-Hadamard rapide (`FHT512`), puis compressée sur `512` bits de signe (`64` octets). Comparer un événement entrant à un catalogue de menaces se ramène alors à `8` instructions matérielles `XOR` + `POPCNT` sur mots de `64` bits.

À travers la base de code de `c2blue55`, le terme **centroïde** désigne quatre objets mathématiques distincts et chaînés :

Objet mathématique	Représentation	Rôle fonctionnel	Implémentation source
1. Centroïde a priori μ_{ref}	Tableau `[512]float32` fixe	Soustrait l'espérance uniforme ($x = x_{\text{raw}} - \mu_{\text{ref}}$)	`internal/engine/feature_extractor.go`
2. Centroïdes menaces RaBitQ	`[8]uint64` (512 bits) dans `CodebookEntry` (72 o)	Veto géométrique `L1b` via $d_H(q,c) \le R_{\text{block}}$ (+ centroïde `L1a`)	`rabitq512.go`, `floppy_engine.go`, `rabitq_probe.go`
3. Centroïdes empiriques μ_B, μ_H	Moyennes `[512]float32` quantifiées en `[512]int8`	Hyperplan séparateur linéaire ($W \propto \mu_H - \mu_B$) en `L1b`	`cmd/c2forge/floppy_builder.go`
4. Centroïde suivi μ_t	Suite `[512]float32` hors ligne	Garde anti-empoisonnement euclidienne + `CUSUM` de Page	`internal/engine/drift_guard.go`

1. Projection 512D et centroïde de référence a priori

Lors de l'évaluation d'un événement `Probe_event_t` sur les trois disquettes générales de tournoi (`floppy_lolbas.c2book`, `floppy_dns_c2.c2book`, `floppy_agent_mcp.c2book`), `FeatureExtractor` résout la charge utile jusqu'à $\text{PageSize} = 4\,096$ octets sans s'interrompre sur les octets `NUL` internes, et peuple un vecteur brut $x_{\text{raw}} \in \mathbb{R}^{512}$ pavé en sept régions contiguës :

$$\dim(x_{\text{raw}}) = \underbrace{256}_{\text{unigrammes}} + \underbrace{128}_{\text{bigrammes}} + \underbrace{64}_{\text{trigrammes}} + \underbrace{16}_{\text{entropies locales}} + \underbrace{12}_{\text{classes ASCII}} + \underbrace{8}_{\text{en-tête}} + \underbrace{28}_{\text{agrégats}} = 512$$

Pour éviter qu'un décalage d'échelle continu ne domine la quantification par signe, `FeatureExtractor.center` soustrait le **centroïde de référence a priori** $\mu_{\text{ref}} \in \mathbb{R}^{512}$ (`buildReferenceCentroid`), défini analytiquement comme l'espérance d'une loi d'octets uniforme sur $\{0, \ldots, 255\}$:

$$x = x_{\text{raw}} - \mu_{\text{ref}}, \qquad \mu_{\text{ref}}[i] = \begin{cases} \frac{1}{256} & \text{si } i \in [0, 255] \quad \text{(fréquences d'unigrammes d'octets uniformes)} \\ \frac{1}{128} & \text{si } i \in [256, 383] \quad \text{(seaux de bigrammes par décalage et XOR)} \\ \frac{1}{64} & \text{si } i \in [384, 447] \quad \text{(seaux de trigrammes à hachage multiplicatif)} \\ 1{,}0 & \text{si } i \in [448, 463] \quad \text{(16 fenêtres d'entropie normalisées par 8 bits)} \\ \frac{\lvert C_{i-464} \rvert}{256} & \text{si } i \in [464, 475] \quad \text{(cardinaux des 12 classes de caractères)} \\ 0 & \text{si } i \in [476, 511] \quad \text{(8 champs d'en-tête et 28 agrégats statistiques)} \end{cases}$$

où les 12 cardinaux $|C_k|$ valent `10` (voyelles), `42` (consonnes), `26` (majuscules), `26` (minuscules), `10` (chiffres), `12` (lettres hexa), `32` (ponctuation), `2` (espace/tabulation), `33` (contrôle), `128` (octets $\ge$ 0x80), `95` (ASCII imprimables) et `62` (alphanumériques). *(Note : les disquettes DNS spécialisées à séparation par familles disjointes `floppy_dns_tunnel.c2book` et `floppy_dns_dga.c2book` utilisent les extracteurs aveugles au domaine parent `ExtractDNSTunnel512` et `ExtractDNSDGA512`, qui centrent directement les descripteurs structurels DNS dans $[-1, 1]^{512}$ avant `QuantizeFHT512`.)*

2. Centroïdes binaires de menaces (RaBitQ FHT512) et rayon de veto

Dans une disquette `.c2book`, chaque attaque réelle de la partition d'apprentissage (ainsi que tout motif hors-distribution isolé par `DiscoverRedhostPatterns`) est compilée dans une entrée `CodebookEntry` de 72 octets : bitcode `[8]uint64` de 64 octets, `ThreatID` (`uint32`), `Subsystem` (`uint16`) et `Severity` (`uint16`).

2.1 Rotation orthonormale de Rademacher–Hadamard (ProjectRabitq)

Une quantification binaire directe sur les axes canoniques écraserait un signal concentré sur quelques coordonnées éparses. `QuantizeFHT512` (`rabitq512.go`) applique donc une rotation orthonormale structurée sans jamais allouer de matrice dense 512×512 :

$$p = \frac{1}{\sqrt{512}} H_{512}\,(s \odot x)$$

- $s \in \{-1, +1\}^{512}$ est un vecteur de signes de Rademacher déterministe engendré par `SplitMix64` à partir de la graine fixe $\text{RabitqSeed} = 0\text{x9E3779B97F4A7C15}$, et $s \odot x$ désigne le produit terme à terme $s_i x_i$.
- H_{512} est la transformation de Walsh-Hadamard rapide en place (`FHT512`), exécutée en $512 \log_2(512) = 4\,608$ additions/soustractions papillon, mise à l'échelle par $\frac{1}{\sqrt{512}} \approx 0{,}044194174$.
- Puisque $Q = \frac{1}{\sqrt{512}} H_{512}\,\mathrm{diag}(s)$ vérifie $Q^\top Q = I_{512}$, l'application est une **isométrie euclidienne exacte** ($\|p\|_2 = \|x\|_2$, $\langle p, p' \rangle = \langle x, x' \rangle$) tout en répartissant uniformément la variance sur les 512 coordonnées.

2.2 Centrage de moyenne et encodage de signe 1-bit (QuantizeRabitq)

Le vecteur tourné $p \in \mathbb{R}^{512}$ est centré par sa moyenne arithmétique avant extraction des bits de signe :

$$\overline{p} = \frac{1}{512} \sum_{j=0}^{511} p_j, \qquad u_j = p_j - \overline{p}, \qquad c_j = \mathbf{1}_{u_j \geq 0} = \begin{cases} 1 \text{ si } p_j - \overline{p} \geq 0 \\ 0 \text{ si } p_j - \overline{p} < 0 \end{cases}$$

Les 512 bits c_j sont rangés en petit-boutiste dans 8 mots `uint64` selon $\mathrm{Bits}\left[\left\lfloor \frac{j}{64} \right\rfloor\right] \leftarrow \mathrm{Bits}\left[\left\lfloor \frac{j}{64} \right\rfloor\right] \text{ OR } (1 \ll (j \bmod 64))$. Lorsqu'une estimation continue de distance euclidienne est requise (`RabitqCode`), les normes centrées $L_1 = \sum_{j=0}^{511} |u_j|$ et $L_2 = \|u\|_2$ relient la distance de Hamming $d_H(q, c)$ à la distance euclidienne carrée par l'estimateur angulaire (`AsymmetricDistanceCodes`) :

$$\cos\hat{\theta}(q, c) = 1 - \frac{2\, d_H(q, c)}{512}, \qquad \hat{d}_2^2(q, c) = \|u_q\|_2^2 + \|u_c\|_2^2 - 2\,\|u_q\|_2\,\|u_c\|_2 \cos\hat{\theta}(q, c)$$

(Un estimateur asymétrique 1-bit × 5-bit par plans de bits transposés sans table est également fourni dans `BitProductScalar` : $\hat{d}^2_{\text{asym}} = \|q\|_2^2 + \|u_c\|_2^2 - 2\|u_c\|_2^2 \langle s_c, q \rangle / \|u_c\|_1$, avec $s_c \in \{-1, +1\}^{512}$.)

2.3 Distance de Hamming déroulée, arbitrage L1a/L1b et rayon calibré R_{block}

À l'exécution (`SearchCentroid`), le bitcode q est comparé à chaque centroïde de menace $c \in \mathcal{C}$ par 8 instructions matérielles `POPCNT` avec arrêt anticipé dès que `maxHamming` est dépassé (`codebookHammingBounded`) :

$$d_H(q, c) = \sum_{w=0}^{7} \mathrm{popcount}(q_w \oplus c_w) \in \{0, \ldots, 512\}, \qquad d_H^*(q) = \min_{c \in \mathcal{C}} d_H(q, c)$$

La monotonie défensive entre `L1a` et `L1b` (`inference_cascade.go`) impose trois règles géométriques :

1. **Subordination de la sonde conforme `L1a`** : `RabitqProbe` mesure les distances de Hamming normalisées moyennes D_1, D_2 vis-à-vis d'au plus 32 prototypes bénins/hostiles et la dispersion Spread_k autour d'un centroïde majoritaire par mot de 64 bits (`classSpread` : le mot w vaut `^uint64(0)` ssi $2\sum_{i=1}^{n} \mathrm{popcount}(\mathrm{proto}_i[w]) \geq 64n$, sinon `0`). Même si `L1a` conclut `ProbeConfirm` bénin, toute libération anticipée est **interdite** dès que $d_H^*(q) \leq \max(R_{\text{block}}, 24)$ hors contexte ontologique certifié (`ontoVerdict != OntoVerdictAllow`).
2. **Veto direct `L1b`** : si $d_H^*(q) \leq R_{\text{block}}$, l'événement reçoit `VerdictBlock` ($Q_8 = 2(128 - d_H^*(q))$), sauf pour un domaine DNS apex enregistrable sans sous-domaine (`isApexDomain`), où `PredictINT8` est évalué paresseusement afin qu'un domaine certifié bénin conforme lève les collisions lexicales.
3. **Quarantaine de bande grise** : si $R_{\text{block}} < d_H^*(q) \leq 24$ (sans certification apex ni autorisation ontologique), l'événement est escaladé au minimum en `VerdictQuarantine`.

À la compilation (`CalibrateBlockRadius`), R_{block} est calibré sur la seule partition bénigne d'apprentissage B_{train} à taux empirique de faux blocages $\leq \alpha_{\max} = 1\%$ (`benignBlockRateMax = 0.01`, plafonné par `FloppyDefaultBlockRadius = 12` et minoré par 1) :

$$R_{\text{block}} = \max\left(\{1\} \cup \left\{ r \in \{0, \ldots, 12\} \;\middle|\; \frac{1}{|B_{\text{train}}|} \sum_{x \in B_{\text{train}}} \mathbf{1}_{d_H^*(q(x)) \leq r} \leq 0{,}01 \right\} \right)$$

3. Centroïdes empiriques de classes et tête linéaire quantifiée INT8

Les événements atteignant L1b sans déclencher de veto par centroïde de menace sont évalués par une tête linéaire signée sur 8 bits (DecisionClass). Dans c2forge (computeEmpiricalHead), les vecteurs de poids implémentent un **discriminant linéaire de Fisher isotrope par centroïdes les plus proches** sur les plongements réels centrés B (bénins, classe 0) et H (hostiles, classe 1) :

$$\mu_B = \frac{1}{|B|}\sum_{x\in B} x \in \mathbb{R}^{512}, \qquad \mu_H = \frac{1}{|H|}\sum_{x\in H} x \in \mathbb{R}^{512}$$

Les poids INT8 $W_0, W_1 \in [-127, 127]^{512}$ (avec $\text{Bias}_0 = \text{Bias}_1 = 0$) quantifient la différence des centroïdes empiriques :

$$W_0[d] = \text{clamp}_{[-127,127]}(\text{trunc}(127 \cdot (\mu_B[d] - \mu_H[d]))), \qquad W_1[d] = \text{clamp}_{[-127,127]}(\text{trunc}(127 \cdot (\mu_H[d] - \mu_B[d])))$$

À l'inférence (PredictINT8), après rejet de toute entrée comportant une coordonnée IEEE-754 NaN ou ±Inf, chaque caractéristique est quantifiée en $q_d = \text{clamp}_{[-127,127]}(\text{trunc}(127 \cdot x_d))$, puis le produit scalaire entier et son score normalisé sont évalués :

$$S_c(x) = \text{Bias}_c + \sum_{d=0}^{511} q_d\, W_c[d], \qquad \tilde{S}_c(x) = \frac{S_c(x)}{127 \times 512} = \frac{S_c(x)}{65\,024}, \qquad c^* = \arg\max_{c\in\{0,1\}} S_c(x)$$

La classe c^* est certifiée **conforme** ssi $\tilde{S}_{c^*}(x) \geq \hat{q}_{c^*}$ (QHat), où computeEmpiricalHead fixe $\hat{q}_c = 0{,}4 \times \overline{\tilde{S}}_c$ si le score normalisé moyen d'apprentissage vérifie $\overline{\tilde{S}}_c > 0{,}02$ (et 0,05 sinon). En L1b, $c^* = 1$ avec conformité produit VerdictBlock ; sans conformité, VerdictQuarantine.

4. Recertification hors ligne et garde anti-empoisonnement CUSUM

En production, les disquettes .c2book sont des projections immuables PROT_READ authentifiées par HMAC-SHA256 (en-tête + corps) et CRC32C de Castagnoli, commutées atomiquement en $O(1)$ via FloppySlot. Le trafic en direct **ne modifie jamais les centroïdes en place**.

Lors des recertifications hors ligne produisant une suite de centroïdes candidats $\mu_0, \mu_1, \ldots, \mu_t \in \mathbb{R}^{512}$, CusumDriftGuard.CheckUpdate (drift_guard.go) prévient tout **empoisonnement graduel** par quatre conditions cumulatives (DefaultDriftGuardConfig) :

1. **Garde de finitude IEEE-754** : rejet immédiat si $\exists d \in \{0, \ldots, 511\}, \mu_t[d] \in \{\text{NaN}, \pm\infty\}$, empêchant toute comparaison flottante non ordonnée de contourner les bornes.
2. **Plafond de déplacement euclidien instantané** ($r_{\max} = 15{,}0$) :

$$\Delta_t = \|\mu_t - \mu_{t-1}\|_2 = \sqrt{\sum_{d=0}^{511} (\mu_t[d] - \mu_{t-1}[d])^2} \leq r_{\max}$$

3. **Budget total de dérive cumulée** ($B = 50{,}0$) :

$$D_t = \sum_{\tau=1}^{t} \Delta_\tau \leq B$$

4. **Détecteur séquentiel CUSUM unilatéral de Page** (tolérance $k = 0{,}1$, seuil d'alarme $h = 5{,}0$, $S_0^+ = 0$) :

$$S_t^+ = \max(0, S_{t-1}^+ + (\Delta_t - k)) \leq h$$

Si l'une de ces inégalités échoue, le candidat μ_t est rejeté, alertCount est incrémenté et le centroïde de référence actif demeure figé sur μ_{t-1}.